

Set Operators in LINQ using C#

In this article, I will discuss **Set Operators in LINQ** using C#. Please read our previous article discussing the **OfType Operators in LINQ** with some examples. As part of this article, we will discuss the following concepts.

1. **What are Set Operators in LINQ?**
2. **Different LINQ Set Operators in C#**
3. **Example to Understand LINQ Set Operators in C#**
4. **When should we use LINQ Set Operators in C#?**

What are Set Operators in LINQ?

The Set Operators in LINQ are used to produce the result set based on the presence and absence of elements within the same or different data sources. That means these operations are performed either on a single data source or multiple data sources, and in the output, some of the data are present, and some of the data are absent.

LINQ Provides set operators to perform mathematical set operations on sequences or collections like arrays, lists, etc. These operators include Distinct, Union, Intersect, and Except. Each operator returns a new sequence of results and can be used with any type that implements the `IEnumerable<T>` interface.

Examples of Set Operations:

Let us discuss some examples where we need to use the set operations.

1. If we need to select distinct records from a data source (No Duplicate Records), then we need to use Set Operators.
2. Suppose we need to select all the Employees of a company except a particular department; then you need to use Set Operations.
3. Another example is that if you have multiple classes and you only want to select all the toppers from all the classes, then you need to use Set Operations.
4. Suppose you have different data sources with similar structures, and you want to combine all the data sources into a single data source, then you need to use Set Operations.

LINQ Set Operators in C#:

LINQ provides several set operators that you can use in your queries:

Distinct

This operator returns distinct elements from a collection, effectively removing duplicates. It's useful when you only want to see each element once.

- **Purpose:** Returns a sequence containing distinct elements from the original sequence, removing duplicates.
- **Use Case:** Use Distinct when you need to eliminate duplicate values from a sequence.

- **Example Scenario:** Removing duplicate names from a list of customer names.
- **Data Cleanup:** Before processing data, you might use Distinct to eliminate duplicates that could skew your results.
- **User Interface:** Populating dropdown menus where you need a list of unique options.

Union

This operator combines two sequences and returns a new sequence that contains the unique elements from both. It's similar to the mathematical union of sets.

- **Purpose:** Produces a sequence that contains the unique elements from both of the two sequences.
- **Use Case:** Use Union when you want to combine two sequences into one sequence that includes elements from both without duplicates.
- **Example Scenario:** Creating a list of all unique products from two different lists of products.
- **Data Aggregation:** Combining data from multiple sources into a single list of unique items.
- **Feature Unification:** When dealing with features from different sets, you want a list representing the full feature set without duplication.

Intersect

This operator returns a new sequence that includes elements that exist in both of the source sequences. It's similar to the mathematical intersection of sets.

- **Purpose:** Creates a sequence that contains the elements that appear in both of the two sequences.
- **Use Case:** Use Intersect to find common elements shared by two sequences.
- **Example Scenario:** Finding common friends between two users in a social network application.
- **Commonality Analysis:** Determining shared characteristics or elements in datasets.
- **Authorization:** Checking for shared permissions between two sets of user roles.

Except

This operator returns the elements from the first sequence that do not exist in the second sequence. It's similar to a subtract operation in sets.

- **Purpose:** Produces a sequence containing elements from the first sequence that do not appear in the second sequence.
- **Use Case:** Use Except to find elements in one sequence but not another.
- **Example Scenario:** Determining which products are in the first list but not in the second, perhaps to identify products that have been discontinued.
- **Set Difference:** When you need to perform a 'set minus' operation to see what's been removed or added between two data collections.
- **Feature Differentiation:** Comparing product features to highlight differences.

Concat:

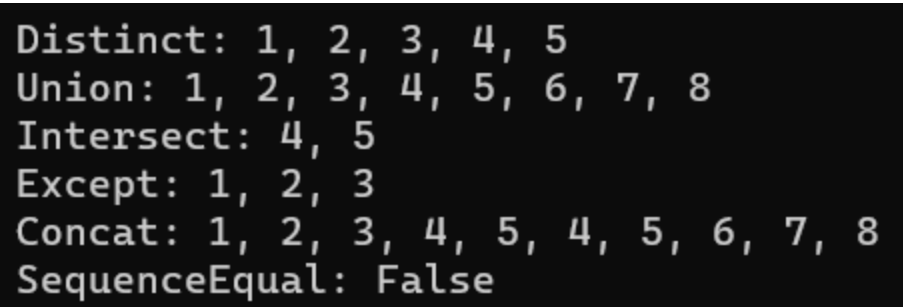
While not strictly a set operation in the mathematical sense, Concat in LINQ is used to append one sequence to another. Unlike Union, Concat does not remove duplicates.

Example to Understand LINQ Set Operators in C#:

The following example shows how to use the Set Operators in C#.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] sequence1 = { 1, 2, 3, 4, 5 };
            int[] sequence2 = { 4, 5, 6, 7, 8 };
            var distinct = sequence1.Distinct();
            Console.WriteLine("Distinct: " + string.Join(", ", distinct));
            var union = sequence1.Union(sequence2);
            Console.WriteLine("Union: " + string.Join(", ", union));
            var intersect = sequence1.Intersect(sequence2);
            Console.WriteLine("Intersect: " + string.Join(", ", intersect));
            var except = sequence1.Except(sequence2);
            Console.WriteLine("Except: " + string.Join(", ", except));
            var concatenated = sequence1.Concat(sequence2);
            Console.WriteLine("Concat: " + string.Join(", ", concatenated));
            bool areEqual = sequence1.SequenceEqual(sequence2);
            Console.WriteLine($"SequenceEqual: {areEqual}");
            Console.ReadKey();
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and white text. It displays the output of the C# program. The output consists of seven lines, each showing a LINQ set operation result. The first line is 'Distinct: 1, 2, 3, 4, 5'. The second line is 'Union: 1, 2, 3, 4, 5, 6, 7, 8'. The third line is 'Intersect: 4, 5'. The fourth line is 'Except: 1, 2, 3'. The fifth line is 'Concat: 1, 2, 3, 4, 5, 4, 5, 6, 7, 8'. The sixth line is 'SequenceEqual: False'.

```
Distinct: 1, 2, 3, 4, 5
Union: 1, 2, 3, 4, 5, 6, 7, 8
Intersect: 4, 5
Except: 1, 2, 3
Concat: 1, 2, 3, 4, 5, 4, 5, 6, 7, 8
SequenceEqual: False
```

Important Points to Note:

- These operations are based on the default equality comparison for the type of elements in the sequence. If you need to compare elements based on a specific property or criteria, you might need to implement a custom `IEqualityComparer<T>`.
- The Union, Intersect, and Except methods all produce sets that do not include duplicate elements. If you need to maintain duplicates, you would need to use concatenation (Concat) and maybe additional filtering.
- The Distinct method does not guarantee that the order of elements is preserved. However, in practice, the order is preserved with the current implementation of LINQ to Objects.

- These operators use deferred execution, meaning the result sequence isn't created until you enumerate.

When should we use LINQ Set Operators in C#?

LINQ set operators are used when you need to perform set-like operations on collections or sequences of data. These operators are useful in various scenarios where you want to work with data sets and perform operations based on set theory principles. Here are some common situations where you should consider using LINQ set operators in C#:

Removing Duplicates: Use the `Distinct` operator when you have a collection with duplicate elements and want to retrieve only the unique elements. This is valuable for generating unique lists or filtering out redundant data.

```
var uniqueNumbers = numbers.Distinct();
```

Combining Data: When you have two or more collections and want to combine them into a single collection while eliminating duplicate elements, the `Union` operator can be helpful.

```
var combinedData = collection1.Union(collection2);
```

Finding Common Elements: If you have multiple collections and you want to find elements that exist in all of them, you can use the `Intersect` operator. This is useful for tasks like identifying shared items across multiple lists.

```
var commonElements = collection1.Intersect(collection2);
```

Identifying Unique Elements: The `Except` operator allows you to find elements that exist in one collection but not in another. This helps find unique or exclusive items in a dataset.

```
var uniqueItems = collection1.Except(collection2);
```

Concatenating Data: When you need to concatenate two or more collections without removing duplicates, the `Concat` operator can be used. This is useful for combining data from various sources.

```
var concatenatedData = collection1.Concat(collection2);
```

Comparing Sequences: The `SequenceEqual` operator helps you compare two sequences to determine if they are identical in terms of the elements they contain and their order. This is useful for verifying whether two lists or sequences match.

```
bool areEqual = sequence1.SequenceEqual(sequence2);
```

Removing Redundant Data: If you have a collection that contains duplicates and you want to eliminate those duplicates, you can use the `Distinct` operator to clean up your data.

```
var uniqueData = data.Distinct();
```

Checking for Data Consistency: When working with data from different sources or systems, you can use set operators to compare data sets and identify inconsistencies or missing elements.

```
var missingData = sourceData.Except(targetData);
```

Filtering Data: Set operators can be used to filter or refine data by comparing it with another dataset and selecting specific elements based on common criteria.

```
var filteredData = allData.Intersect(criteriaData);
```

In the next article, I will discuss the [LINQ Distinct Method in C#](#) with Examples. In this article, I explain the need for Set Operators and the different types of Set Operators Provided by LINQ.